

01. システム概要説明（たたき台）

- ドキュメント種別: 上流工程 / システム概要・要件定義のアウトライン
- 対象システム（仮称）: 居酒屋店舗システム（SaaS型） / AIネイティブ再構築版
- 作成日: 2026-09-03
- ステータス: フェーズ1向け凍結（2026-09-09）。以降の変更はフェーズ1スコープ内の誤り訂正・実装スパイク結果の反映に限る。

本書は、以前「一人SaaSシステム」として作成した「居酒屋店舗システム」を参考に、AIネイティブなアプローチでシステムを新規構築（再構築）するための概要と要件定義の出発点をまとめたものです。詳細な機能仕様・画面設計・API設計は後続ドキュメントで扱います。

1. 背景と目的

1.1 背景

- 個人～小規模で運営される居酒屋・飲食店では、予約管理・注文・会計・仕入れ・シフト・売上把握などの業務が、紙・電話・表計算ソフト・複数の単機能アプリに分散している。
- 既存の「一人SaaSシステム（居酒屋店舗システム）」は、1人（または少人数）の開発・運用を前提に、必要最小限の機能をSaaSとして提供する構成だった。
- 近年はLLM（大規模言語モデル）を前提に設計する「AIネイティブ」なアプローチが可能になり、「機能をメニューから選んで操作する」だけでなく「自然言語で指示・相談する」「AIが先回りして提案・自動化する」体験を中心に据えた再設計が現実的になった。

1.2 このシステムで解決したい課題

#	課題	現状	目指す姿
1	業務ツールの分散	予約帳・レジ・仕入れメモ・シフト表がバラバラ	1つのSaaSに統合
2	属人化・経験依存	発注量やシフトが「勘」頼み	売上・来客データに基づくAI提案
3	入力・集計の手間	日次売上や在庫を手集計	自動集計・自然言語での質問応答
4	分析ができない	データはあるが活用されていない	ダッシュボード+AIによる示唆出し
5	導入・運用コスト	専用端末・オンプレは高価	ブラウザ完結・月額サブスクリプション

1.3 ゴール（初期リリースの狙い）

- 1店舗の経営管理者が「予約受付 → 来店 → 注文 → 会計 → 日次締め」までを本システムだけで回せる。

- 仕入れ・在庫・シフト・売上分析を同じ画面で扱える。
- AIアシスタントに日本語で質問・依頼するだけで、集計・提案・下書き作成ができる。
- SaaSとして複数店舗（複数テナント）へ横展開できる基盤を最初から持つ。

2. システムコンセプト

2.1 提供形態

- **SaaS型（マルチテナント）**: 店舗（企業）ごとにデータを論理分離。ブラウザからログインして利用。
- **サブスクリプション課金**: 店舗数・機能プランに応じた月額課金を想定。
- **マルチデバイス**: PC（管理業務）、タブレット（ホールでの注文・卓管理）、スマートフォン（経営管理者の外出先確認、スタッフのシフト確認、顧客の予約）。

2.2 「AIネイティブ」の意味（本プロジェクトでの定義）

従来型（AIは後付けの一機能）ではなく、以下を設計の中心に置く。

1. **会話が主操作の一つ**: どの画面からでもAIアシスタントを呼び出し、日本語で「今週末の予約状況は？」「先月より粗利が落ちた原因は？」「発注書のたたきを作って」と依頼できる。
2. **先回りの提案（プロアクティブ）**: AIが売上・在庫・天気・予約から「明日の仕込み量」「発注推奨」「シフト過不足」を通知する。
3. **非構造データの構造化**: 手書きメモ・レシート画像・電話メモ・口コミを、AIがテキスト化・分類・要約。
4. **人間が最終承認**: AIの提案は必ず下書き扱い。発注確定・シフト確定・送信は人が承認する（Human-in-the-loop）。
5. **監査可能**: AIの入力・出力・利用データを記録し、後から根拠を追跡できる。

3. ターゲットユーザー

3.1 想定顧客セグメント

- 個人経営～数店舗規模の居酒屋・大衆酒場・ダイニングバー。
- 「専任の店長／事務員を置けない」「経営管理者が現場も経理も見る」規模。
- 将来的には小規模チェーン（5～20店舗）や、フランチャイズ本部にも拡張可能。

3.2 利用者ロール（ペルソナ）

ロール	主な関心事	よく使う機能	主な利用デバイス
経営管理者	売上・利益、コスト、全体把握	ダッシュボード、売上分析、AI相談、発注承認	PC / スマホ
店長 / ホール責任者	当日オペレーション、予約、人員	予約管理、卓・注文管理、シフト、在庫	タブレット / PC
ホールスタッフ	注文取り、配膳、会計補助	注文入力（ハンディ）、卓状況、会計	タブレット / スマホ

キッチンス タッフ	調理指示、品切れ 管理	キッチンディスプレ イ、在庫の品切れ登録	タブレット（KDS）
アルバイト 全般	自分のシフト、勤 怠	シフト確認、希望提 出、打刻	スマホ
顧客（来店 客）	予約、待ち時間、 メニュー確認	Web予約、モバイル オーダー（任意）、順 番待ち受付	スマホ
SaaS運営者 （自社）	テナント管理、課 金、稼働監視	管理コンソール、契 約・請求、サポート、 利用状況	PC

4. 主要機能一覧（アウトライン）

優先度は初期案。◎=初期リリース必須 / ○=第2フェーズ / △=将来構想

4.1 予約・来店管理

- 予約台帳（日／週表示、電話・Web・当日ウォークインの一元管理）◎
- Web予約フォーム（コース選択、人数、アレルギー・要望欄）◎
- 席（テーブル）マスタと空席・稼働状況の可視化◎
- 順番待ち（ウェイティング）受付・呼び出し（SMS/LINE通知）○
- 予約リマインド・無断キャンセル対策（前日確認、デポジット）○
- 常連客の来店履歴・好み・NG事項の表示○

4.2 注文・卓管理（ホールオペレーション）

- 卓ごとの注文入力（ハンディ／タブレット）、追加注文、取消・数量変更◎
- コース・飲み放題の時間管理（ラストオーダー通知）◎
- キッチンへの注文連携（キッチンディスプレイ or 伝票印刷）◎
- モバイルオーダー（客の私物スマホで卓上QRを読み取り→注文画面を起動して注文）◎
 - 会計はレジで行う（フェーズ1では客のスマホ上での決済は行わない）
 - スタッフのハンディ入力と同じ注文・卓管理へ合流させる
- 卓上に専用タブレット端末を設置しての注文△（フェーズ3以降の拡張オプション）
- 卓の結合・分割、席移動○

4.3 会計・レジ（POS）

- 会計（割り勘、クーポン、値引き、混合支払い＝1会計で複数手段）◎
- 伝票・レシート発行、領収書発行◎
- 日次締め（レジ点検・精算）、支払手段別の内訳集計、売上日報の自動生成◎
- インボイス制度対応（適格請求書、税区分）◎
- 会計ソフト連携（freee / マネーフォワード等へのエクスポート）○

4.3.1 決済手段（店舗ごとに選択）

- **方針:** システムは特定の決済代行に依存せず、**決済手段プラグイン方式**とする。店舗ごとに、対応可能な手段を複数選択して有効化できる。各手段は接続情報（加盟店ID、APIキー、ウォレットアドレス等）を店舗設定として持つ。

- **対応手段（初期スコープ）**：現金／PayPay／楽天ペイ／クレジットカード／JPYC
- 会計時は有効化された手段のみ選択肢に表示。1会計で複数手段の混合支払いに対応。
- 決済結果（成功／失敗／金額／取引ID）を記録し、日次締めで手段別に自動集計。API連携できない手段は金額手入力での消し込みを許容する。

手段	連携方式（想定）	優先度	備考
現金	外部サービス不要。POS内で管理（釣銭・レジ締め）	◎	—
PayPay	PayPay加盟店契約を直接（Merchant Payment API）。動的QR（ユーザースキャン方式：本システムが金額入りQRを発行し顧客がスキャン）・ストアスキャン方式（顧客が提示するバーコードを店舗側で読み取り決済）のいずれも同じ公開APIで自社開発システムから直接呼び出せ、結果はWebhookで取得	◎	直接契約が手数料・連携の両面で有利。QR決済シェア国内最大。フェーズ1は動的QR（ユーザースキャン方式）を採用（02_requirements.md FR-G06）
クレジットカード	Square（Mobile Payments SDK等）を第一候補。自社POS作り込み方針ならStripe Terminal	◎	リーダーを会計画面から起動し、結果をAPI/Webhookで取得
楽天ペイ	店舗提示・QR表示（静的／動的）、または顧客提示・ストアスキャンのいずれでも運用可能。QRの生成・表示自体は無料の「楽天ペイ実店舗アプリ」を店舗の普通のスマホ／タブレットに入れるだけででき、専用ハードウェアは不要。ただし決済結果を本システムに 自動反映 させるには「楽天ペイターミナル」という専用ハードウェア＋楽天ペイが個別に提携した特定のレジ製品（2026-09時点：スマレジ／ユビレジ／funfo／Aiony／poscube／FIT SHOP）である必要があり、本システムのような自社開発システムは現時点で対象外。フェーズ1は金額手入力・目視確認でPOSに記録し、自動連携の可否・費用はフェーズ2で検討	◎	既存店舗の運用（QR表示／静的QR／ストアスキャンのいずれでも）をそのまま継続できる
JPYC	当面は店舗ウォレットのQR表示＋オンチェーン着金確認で最小実装。将来的にJPYC対応決済ゲートウェイ	△	円建てステーブルコイン。飲食店向け決済インフラは未整備のためオプション扱い。会計・税務（受領時の扱い、円転タイミング）は要検討

- **将来の拡張**：Airペイ／stera など「1台でクレカ＋各種QR＋交通系」をまとめる 決済代行ア

アダプタを後発で追加できる構造にしておく。

- **注意:** 各サービスの手数料率・対応手段・JPYCの提供状況は変動するため、要件定義フェーズで最新情報を確認する（→8章の論点）。
- **検討メモ：複数ブランド対応のストアスキャン端末（2026-09時点、未決）：** 実店舗で「PayPayのロゴが付いた端末なのに楽天ペイの読み取りにも対応していた」という観察報告あり。これは特定ブランド専用の機械ではなく、決済代行（仲介）会社1社と契約すると、その会社の端末1台で複数のQR決済ブランドをまとめてスキャン・処理できる、日本でよくある方式だと考えられる（上記「将来の拡張」のAirペイ／stera等と同系統）。
 - 現状の方針（上表）は、PayPayのみ直接契約＋自動連携（Webhook）、楽天ペイはフェーズ1ではシステム連携なしの手動確認、という**ブランドごとの個別対応**。決済手段プラグイン方式（04_architecture.md §7.1）で1手段＝1アダプタとして作っているため、後からこの「複数ブランドをまとめる仲介会社」を新しいアダプタとして追加すること自体は構造上可能。
 - 乗り換えを具体的に検討する際に確認が必要な点：(1) その仲介会社が支払い完了をAPI/Webhookで自動通知してくれるか（できなければ楽天ペイと同じ手動入力運用になる）、(2) 手数料・契約条件（02_requirements.md 11.1の経営判断と同じ枠組み）。
 - **補足（上記(1)の掘り下げ）：** 仲介会社が用意した専用端末でお客様のスマホのQR／バーコードを読み取って決済する場合も、「端末→PayPay・楽天ペイ」の連携は仲介会社の契約で完結するだけで、「仲介会社の端末→この店舗システム」を繋ぐ仕組みはまだ何も実装していない。したがって、
 1. の自動通知（API/Webhook）を仲介会社が提供し、こちらでそれを受け取るアダプタを実装しない限り、仲介会社の端末で決済が完了しても店舗システムには自動反映されず、楽天ペイと同じ「スタッフが仲介会社端末の画面を目視確認し、店舗システムに金額を手入力する」運用になる。
 - まだどの仲介会社を使うかの検討・意思決定はしていない。この段落は「こういう選択肢がある」という記録のみで、フェーズ1の実装方針（上表）を変更するものではない。
 - **追記（2026-09-15、楽天ペイの動的QR方式について確認）：** 楽天ペイ公式サポートサイトで確認したところ、楽天ペイにも「QR表示方式（動的QR）」——店舗が楽天ペイ店舗アプリに金額を入力してQRを生成し、顧客がそれをスキャンする、PayPayの動的QRと同じ仕組み——が存在する。したがって上表の「静的QR」という表現は不正確で、楽天ペイが選べるのはQR表示（静的／動的）とストアスキャンのどちらもという点を修正した。ただし、動的QRが使えることと、その決済結果を本システムに自動で反映できることは別の話であり、楽天ペイ側の自動連携（POSレジ連携）は「楽天ペイ ターミナル」という専用ハードウェアの導入とPOS側の提携アプリでのペアリングが前提になっている模様で、PayPayのように契約さえすれば使える公開APIとは条件が異なる。フェーズ1を手動運用にする結論自体は変わらず、その理由づけのみをこの追記の内容に差し替える。
 - **追記2（2026-09-15、PayPay／楽天ペイの自動連携できる範囲を比較）：** ここまでの調査で、「動的QRを表示できるか」と「決済結果を自社システムに自動で反映できるか」は別の話だと分かった。整理すると次の通り。
 - **PayPay：** 動的QR（ユーザースキャン方式）・ストアスキャン方式のどちらも、PayPayが公開している 開発者向けAPIをそのまま使えば、本システム自身が直接QRを発行し、決済結果もWebhookで直接受け取れる。project6thが何か特別な提携をする必要はなく、PayPayに開発者登録して審査に通ればよい。
 - **楽天ペイ：** QRの表示自体は「楽天ペイ実店舗アプリ」（無料・普通のスマホ／タブレットで動く）で専用ハードウェアなしにできる。しかし決済結果を自動

で本システムに反映させる機能は、「楽天ペイ ターミナル」という専用ハードウェアの導入に加えて、楽天ペイが個別に提携している特定のレジ製品（2026-09時点：スマレジ／ユビレジ／funfo／Aiony／poscube／FIT SHOP）でなければ使えない。これは PayPayのような「誰でも申請すれば使える公開API」ではなく、「楽天ペイが1社ずつ個別に契約して作った専用の連携」であり、project6thはこのリストに入っていないため、今のところ自動反映機能は利用できない。project6thがこれを使うには、(a) 楽天ペイと個別に交渉して新しい提携先になる、または (b) 既存の提携レジ製品を自社システムの代わりに導入し、そちらと連携する、のいずれかが必要になる見込み（詳細な条件・費用は要問い合わせ）。

- **結論**：本システムをそのまま使い続けながら決済結果を自動連携できるのは PayPay側だけであり、決済手段プラグイン方式の中でPayPayを優先し、動的QR（ユーザースキャン方式）を軸に実装するという上表・FR-G06の方針を維持する。楽天ペイはフェーズ1どおり手動確認のまま据え置く。
- **追記3（2026-09-15、楽天ペイ・フェーズ1の実際の運用フローを整理）**：「決済結果を自動反映できない」場合の現場オペレーションを具体的に整理すると、次の2段階になる。
 1. QRを表示している「楽天ペイ実店舗アプリ」（店舗側の端末）自体が、決済完了時に完了画面を表示するので、スタッフはそれを見て支払い完了を確認する（顧客のスマホ画面の確認は補助的な手段であり、主な確認先は店舗側アプリの画面）。
 2. 本システム（project6th）にはこの完了が自動で伝わらないため、スタッフが本システムの会計画面に戻り、「完了」等のボタンを押して手動で記録する。つまり「楽天ペイ実店舗アプリでの確認」と「本システムでの手動記録」という、別々の2つの操作をスタッフが両方行う必要がある、という点がフェーズ1運用の実務上のポイント（FR-G07b）。

4.4 メニュー管理

- メニュー／カテゴリ／価格／写真／提供時間帯の管理 ◎
- 期間限定メニュー、売り切れ・提供停止の即時反映 ◎
- 原価・売価・粗利率の登録とレシピ（構成材料）定義 ○（フェーズ2以降）
- 多言語メニュー表示（訪日客向け） △

4.5 在庫・仕入れ管理

- 食材・ドリンクの在庫マスタ、単位・在庫数・発注点 ○（フェーズ2以降）
- 入荷・出庫・棚卸し・廃棄ロスの記録 ○（フェーズ2以降）
- 仕入先マスタ、発注書の作成・送信（メール／FAX代替） ○
- レシピ連動の理論在庫と実在庫の差異（棚卸差異）分析 ○
- レシート・納品書の画像取り込み → 明細の自動読み取り（AI-OCR） ○

4.6 スタッフ・シフト・勤怠

- スタッフマスタ（役割、時給、スキル、勤務可能時間） ◎
- シフト希望提出 → シフト表作成 → 公開・通知 ◎
- 打刻（勤怠）、労働時間集計、人件費の見える化 ◎
- 法令チェック（休憩、深夜、連続勤務などのアラート） ○
- 給与計算ソフト向けエクスポート ○

4.7 顧客管理（CRM）／販促

- 顧客台帳（連絡先、来店回数、累計利用額、メモ、同意管理）○
- 会員・ポイント・スタンプカード（デジタル）○
- LINE公式アカウント連携、クーポン配信、誕生日DM △
- 口コミ（Google/食べログ等）の収集・要約・返信下書き △

4.8 売上分析・レポート／ダッシュボード

- 売上日報（日次締め連動、当日の売上・客数・客単価・決済手段別内訳）※フェーズ1はこれで代替◎（→4.3）
- 日次／週次／月次の売上、客数、客単価、回転率○（フェーズ2以降）
- 時間帯別・曜日別・席別・メニュー別の売上と粗利○（フェーズ2以降）
- ABC分析（売れ筋・死に筋）、フードコスト率、人件費率（FL比率）○
- 予算対比、前年同月比、目標進捗○
- 経営ダッシュボード（KPIカード＋グラフ＋AIコメント）○（フェーズ2以降）

フェーズ1ではダッシュボード画面を作らず、日次締めが生成する売上日報で数字を確認する。ただし後から分析・ダッシュボードを載せられるよう、取引データは明細粒度・イベント単位で保持する（→7章フェーズ1の注記、8章）。

4.9 AIネイティブ機能

フェーズ1では実運用での使用頻度が低いと見込まれるため、AI機能はフェーズ2以降に開始する。フェーズ1は「AIを後付けできる状態」を作ることに注力する（取引データの明細粒度保持、AI入出力を残せる監査ログの器、AIオーケストレーション層の設計余地）。

- **AIアシスタント（チャット）**：全画面から呼び出し、日本語で集計・検索・下書き作成○（フェーズ2以降）
 - 例：「先月の金曜の平均客単価は？」「ハイボールの原価率が高い理由を説明して」
- **自然言語レポート**：ダッシュボードの数字に、変化の要因や着眼点を文章で自動注釈○（フェーズ2以降）
- **需要予測**：曜日・季節・天気・予約・イベントから来客数／売上を予測○
- **自動発注提案**：予測来客数×レシピ×理論在庫から発注量を算出し、発注書を下書き○
- **仕込み量の提案**：翌日の仕込みリストを提示○
- **シフト最適化提案**：予測来客数に対する必要人員とシフト案を提示○
- **メニュー・価格の提案**：粗利・売れ行き・トレンドから改定案やセット提案 △
- **画像・音声入力**：レシート/納品書のOCR、電話メモの文字起こし・要約○
- **口コミ要約と返信下書き**：評価の傾向分析と、店舗トーンに合わせた返信案 △
- **異常検知**：売上急落、廃棄ロス増加、レジ差異などをアラート○

すべてのAI出力は「下書き・提案」であり、確定操作（発注送信、シフト公開、顧客への配信、会計）は人間の承認を必須とする。

4.10 SaaS基盤・管理機能

- テナント（店舗／企業）管理、店舗の複数拠点対応◎
- ユーザー・ロール・権限（RBAC）、パスワードリセット、2要素認証◎
- 契約プラン・課金・請求（Stripe等）、トライアル、利用量メータリング○

- 監査ログ（誰が何をしたか、AIの入出力履歴）◎
- データエクスポート／バックアップ、退会時のデータ削除◎
- 通知基盤（メール、Web Push、LINE、SMS）○
- 運営者向け管理コンソール（テナント一覧、サポート、稼働監視）○

5. システム構成案（アーキテクチャのたたき台）

5.1 全体像

[利用者]

- └ 経営管理者/店長 … PCブラウザ（管理・分析）
- └ ホール/キッチン … タブレット（注文・卓・KDS）… 店が用意する業務端末
- └ スタッフ … スマホ（シフト・勤怠）
- └ 顧客 … 自分のスマホ
 - └ Web予約（席予約）…………… フェーズ1
 - └ モバイルオーダー（卓上QRを読み取り→スマホで注文） フェーズ1
 - └ 順番待ち（当日の空席待ち登録・呼び出し）…………… フェーズ2以降
- └ （卓上に専用タブレット端末を置く注文方式）…………… フェーズ3以降の拡張
 - └ HTTPS



[フロントエンド（SPA / React + TypeScript + Vite）]

- 管理コンソール / ホール端末UI / 顧客向け予約サイト
 - └ REST / JSON（将来 WebSocket for リアルタイム卓状況）



[バックエンド（Spring Boot / Java 21）]

- 認証・認可（JWT + RBAC、マルチテナント境界の強制）
- 業務API（予約・注文・会計・在庫・シフト・分析）
- AIオーケストレーション層（プロンプト構築 / ツール実行 / ガードレール）



[RDB: PostgreSQL]

- 業務データ本体
- テナント分離
- 監査ログ

[AI / LLM 基盤]

- LLM API（例: Claude API）
- RAG用の埋め込み検索（pgvector 等）
- バッチ予測（需要予測・発注）



[外部連携]

決済（店舗ごとに選択: PayPay / クレカ決済代行 / 楽天ペイ (QR表示: 静的・動的 / ストアスキャン) / JPYC ほか、→ 4.3.1)
/ 会計 (freee 等) / LINE / SMS / 天気API / メール

- 顧客がスマホで行う操作は「Web予約」「モバイルオーダー（卓上QR）」「順番待ち」の3種類。フェーズによって開放範囲が異なる（上図の注記）。いずれも客の私物スマホ+ブラウザで完結させる。卓上に専用タブレット端末を設置する注文方式は、導入・保守コストが大きいのでフェーズ3以降の拡張オプションとする。

5.2 フロントエンド

- **技術:** React 19 + TypeScript + Vite（既存資産を継承）、ルーティングは React Router。
- **アプリの分け方（案）:**
 - admin（経営管理者・店長向け: 分析、マスタ管理、AI相談）
 - pos（ホール／キッチン向け: 注文・卓・会計・KDS。オフライン耐性を重視）
 - guest（顧客向け: Web予約〈フェーズ1〉、モバイルオーダー＝卓上QR〈フェーズ1〉、順番待ち〈フェーズ2〉。客の私物スマホのブラウザで動作）
- **UI方針:** コンポーネントライブラリ導入（例: MUI / shadcn 相当）で開発を高速化。ホール端末はタッチ操作前提の大きめUI。
- **状態管理・通信:** サーバ状態は TanStack Query 等、リアルタイム卓状況は WebSocket / SSE。
- **AIアシスタントUI:** どの画面からも開けるサイドパネル。結果は表・グラフ・「承認」ボタン付きの下書きで返す。

5.3 バックエンド

- **技術:** Spring Boot 4.x / Java 21（既存資産を継承）、ビルドは Maven。
- **API:** REST（OpenAPI で仕様管理）。将来的に一部 GraphQL や WebSocket を検討。
- **レイヤ構成:** controller / service / repository（既存構成を踏襲）+ ai（AI連携）モジュール。
- **認証・認可:**
 - ログイン画面の入力はメール＋パスワード（将来 SSO / 2FA）。テナントはURLサブドメイン（<company_code>.<サービスドメイン>）で指定し、ユーザーの一意キーは company_id + email（詳細は 04_architecture.md §6.1）。
 - ログイン前はサブドメイン由来のサーバ側セッションでテナントを保持、ログイン後は JWT ベース、ロールベースアクセス制御（RBAC）。
 - すべてのクエリでテナント（店舗）境界を強制（後述）。
- **AIオーケストレーション層の責務:**
 - ユーザーの依頼＋必要な業務データを集めてプロンプトを構築（RAG）。
 - LLMに「使える操作（ツール）」を定義して渡す（例: 売上集計, 在庫照会, 発注書ドラフト作成）。
 - 出力の検証・整形、機密情報のマスキング、AI入出力の監査ログ記録。
 - 確定操作は必ず「承認待ち」状態で保存し、人の操作でコミット。
- **バッチ処理:** 日次で需要予測・発注提案・異常検知を実行（Spring Batch / スケジューラ）。

5.4 データベース

- **DBMS:** PostgreSQL（既存資産・開発環境を継承。WSL内 localhost:5432 で開発）。
- **マルチテナント方式（案）:**
 - 初期は **共有DB・共有スキーマ + company_code（テナントID）列** で論理分離（低コスト）。
 - すべてのテーブルにテナントIDを持たせ、アプリ層（と可能なら Row Level Security）で分離を担保。
 - 規模拡大時に「テナントごとスキーマ分離」や「大口テナントのDB分離」へ移行できる設計にしておく。
- **主なエンティティ（初期の想定・詳細は別ドキュメント）:**
 - テナント/店舗: company(店舗), store_location（多拠点）, user, role
 - 予約・席: reservation, table(席), waitlist
 - 注文・会計: order, order_item, check(伝票), payment
 - メニュー: menu_item, menu_category, recipe, recipe_ingredient

- 在庫・仕入れ: ingredient, stock_movement, supplier, purchase_order, purchase_order_item
 - スタッフ: staff, shift_request, shift, time_clock
 - 顧客: customer, visit_history, loyalty_point
 - 分析・AI: daily_sales_summary, ai_conversation, ai_message, ai_suggestion, audit_log
 - 埋め込み検索: document_embedding (pgvector。マニュアルや過去レポートのRAG用)
- **監査ログ**: 業務操作とAI入出力を追記専用で保存。

5.5 AI / LLM 基盤

- **LLM**: Claude API を第一候補（最新の Claude 5 系。用途に応じて上位/軽量モデルを使い分け）。抽象化レイヤを挟み、モデル差し替え可能にする。
- **RAG**: 店舗のマスタ・過去日報・マニュアル・FAQ を埋め込み、質問時に関連情報を注入。ベクトルストアは PostgreSQL + pgvector から開始。
- **ツール実行 (function calling)** : LLMが呼べる社内APIを限定的に定義。危険操作は「提案のみ」。
- **予測モデル**: 需要予測などは、初期はシンプルな統計／勾配ブースティング等でも可。データ蓄積後に高度化。天気・カレンダー（祝日・地域イベント）を特徴量に。
- **ガードレール**: 入力/出力のPIIマスキング、プロンプトインジェクション対策、コスト・レート制限、ハルシネーション対策（数値は必ずDB集計結果を根拠に提示）。

5.6 インフラ・非機能（初期案）

- **実行環境**: コンテナ（Docker）。開発は WSL2 上、本番はクラウド（後日選定）。
- **CI/CD**: GitHub Actions でテスト・ビルド・デプロイ。
- **監視・ログ**: アプリログ集約、稼働監視、エラートラッキング、AI利用量・コストの可視化。
- **バックアップ**: DBの定期バックアップとリストア手順、退会時のデータ削除フロー。
- **セキュリティ**: HTTPS必須、パスワードハッシュ化、権限最小化、決済情報は自社で保持せず決済代行に委譲（PCI DSS範囲の最小化）。
- **可用性目標（仮）**: 営業時間帯（夕方～深夜）のダウンタイムを最小化。オフライン時もホール端末で注文入力を継続でき、復旧後に同期。

6. 非機能要件の観点（洗い出し用）

分類	論点（例）
性能	ピーク時（金曜夜）に複数卓の同時注文、会計のレスポンス
可用性	営業時間中の障害許容、オフライン注文の継続
拡張性	テナント増加、多拠点、将来のチェーン対応
セキュリティ	テナント分離、権限管理、監査、個人情報保護法対応、同意管理
保守性	「一人（少人数）で運用」を前提にした運用自動化・可観測性
コスト	LLM API 利用料、インフラ費、決済手数料をプランに織り込む

法令	インボイス制度、労働時間管理、特定商取引法（Web予約）、景表法（販促）
国際化	訪日客向け多言語（将来）
AI固有	出力の説明可能性、誤りの影響範囲、承認フロー、学習データの扱い

7. スcopeとフェーズ分け（提案）

フェーズ1（MVP / 初期リリース）

- 認証・テナント/店舗・ユーザー管理（既存の登録・ログインを土台に拡張）
- 予約台帳+Web予約フォーム
- メニュー管理（メニュー／カテゴリ／価格／写真。原価・レシピはフェーズ2以降）
- 注文・卓管理・会計（POS）・日次締め
- モバイルオーダー（客の私物スマホで卓上QR→注文。スタッフのハンディ入力と同じ卓管理に合流。会計はレジ）
- 決済手段：現金+PayPay+クレジットカード+楽天ペイ（店舗ごとに選択）、混合支払い、手段別締め。楽天ペイは店舗提示・QR表示（静的／動的）、または顧客提示・ストアスキャン（楽天ペイ実店舗アプリ等）+金額手入力・目視確認の運用（システム連携なし）
- シフト・勤怠の基本
- 売上日報（日次締め連動）※凝ったダッシュボードはフェーズ2
- 監査ログ
- （後付け前提の土台）取引データを明細粒度・イベント単位で保持し、AI入出力を残せる 監査ログの器と、AIオーケストレーション層を差し込める構造だけ用意する。経営ダッシュボード・AIアシスタントの画面自体はフェーズ1では作らない。

フェーズ2

- 経営ダッシュボード（KPIカード+グラフ）、時間帯別・曜日別・メニュー別などの集計分析
- AIアシスタント（集計・検索・自然言語レポート・下書き作成）
- 在庫の基本管理（在庫マスタ・発注点、入在庫・棚卸し・廃棄ロスの記録）
- メニューの原価・売価・粗利率、レシピ（構成材料）定義
- 需要予測・自動発注提案・仕込み提案・シフト最適化提案
- 仕入先・発注書、AI-OCR（レシート/納品書）
- CRM・ポイント・LINE連携・販促
 - 顧客（ゲスト）の会員登録・テナント単位でのログイン、過去の予約履歴の閲覧、ログイン状態からの予約作成
- 決済手段の追加：JPYC、決済代行アダプタ（Airペイ／stera等）、楽天ペイの決済API自動連携、会計ソフト連携
- 順番待ち、リマインド、無断キャンセル対策
- SaaS課金・プラン・運営者コンソール

フェーズ3（将来構想）

- 卓上に専用タブレット端末を設置しての注文方式、口コミ要約・返信、価格最適化

- 小規模チェーン／FC本部向け機能（拠点横断分析、標準レシピ配布）
 - 多言語対応、外部BI連携、API公開
-

8. 検討事項・オープンな論点

- 参考にする既存「一人SaaS 居酒屋店舗システム」から、**再利用する資産と作り直す部分の線引き**（現状はSpring Boot+React+PostgreSQLの雛形とユーザー管理のみ）。
 - マルチテナント分離のレベル（共有スキーマ+テナントID で始めてよいか）。
 - ホール端末のオフライン要件をどこまで厳格にするか。
 - LLMの利用範囲とコスト上限、どの操作まで自動化を許すか（承認フローの設計）。
 - 決済・会計・LINE など外部サービスの選定タイミング。
 - 決済手段ごとの連携方式・手数料・契約主体（店舗が個別契約か、当社が一括契約して再委託か）、PayPay直接契約／クレカ決済代行（Square 等）／JPYC の着金確認方式の確定（→ 4.3.1）。
 - JPYC 受領時の会計・税務処理と、即時円転するかどうかの方針。
 - 初期ターゲットを「1店舗経営管理者」に絞るか、最初から複数店舗を見据えるか。
 - 分析・AIをフェーズ2で後付けするための前提（フェーズ1で取引データを明細粒度・イベント単位で保持し、日次締めで明細を捨てない）を、03_domain_model.md で確定させる。
 - フェーズ1のモバイルオーダーの設計論点：卓とQRの紐付け（テーブルごとの固定QR）、ログイン無しでの卓セッション管理、注文の悪用・いたずら対策（数量上限、要スタッフ確認品）、ラストオーダー時刻の制御、品切れ・提供停止のリアルタイム反映、アレルギー・要望の入力、同一卓で複数人が同時に注文する場合の整合、スタッフ入力との合流方法。
-

9. 次のステップ

1. 本書のレビュー（機能の過不足、優先度、フェーズ分けの合意）。
2. 02_requirements.md（要件定義書）：ユースケース、機能要件の詳細、非機能要件の数値目標。
3. 03_domain_model.md: エンティティ・ER図・状態遷移（予約／注文／発注のライフサイクル）。
4. 04_architecture.md: 構成の確定、API一覧、AIオーケストレーション詳細、テナント分離方式。
5. フェーズ1のバックログ化と見積り。